From 69 Characters to 12 Keys: A Corpus-Driven Flick Keyboard for One-Handed Thai Mobile Input

Chawabhon Netisingha
Independent Researcher
Jn03official@gmail.com

Abstract

Thai script requires a repertoire of 69 characters—44 consonants, 21 vowel and diphthong forms, and 4 tone marks—yet mobile Thai input still relies on miniaturized QWERTY layouts originally designed for two-handed desktop typing. We present **PadPim**, to our knowledge the first frequency-optimized 12-key flick keyboard for Thai, adapting the Japanese *kana* flick-input paradigm to Thai script. We describe a complete data-driven pipeline: character frequency analysis from a corpus of over one million lines of Thai conversation collected from 138 university students, a position-scoring model that combines grid ergonomics with flick-direction ease of use, and a greedy optimization algorithm that assigns 57 characters to 12 keys with 5 flick directions each. The 57 main-page characters and 3 space-key tone marks together cover approximately **97.5%** of Thai character occurrences; only 9 characters (∼0.1% of text) require a shift layer. We present two layout variants—*PadPim-Opti* (pure frequency optimization) and *PadPim-Key* (vowel-grouped for learnability)—and compare them against the Kedmanee and Pattachote QWERTY baselines on theoretical efficiency metrics including keys-per-character, touch-target area, and shift frequency. PadPim is implemented as an open-source Android input-method editor and is available for public beta testing.

Keywords: Thai keyboard, flick input, mobile text entry, keyboard layout optimization, one-handed input, character frequency analysis

1. Introduction

Mobile text entry in Thai poses a unique challenge. The Thai script uses 69 distinct characters: 44 consonants, 21 vowel and diphthong forms (some written before, above, below, or after the base consonant), and 4 tone marks that alter pronunciation [1]. The standard *Kedmanee* keyboard layout, designed in 1968 for mechanical typewriters, maps these characters onto 47 keys spread across two shift layers. On a modern smartphone, those 47 keys shrink to touch targets as small as 6 mm wide—well below the 9.2 mm recommended minimum for reliable thumb input [3]. Users must frequently toggle shift to access common vowels and tone marks, adding cognitive load and motor overhead.

In Japan, a radically different solution has become the dominant mobile input method: the **12-key flick keyboard**. Instead of replicating the full desktop layout, Japanese mobile input groups the 46 hiragana characters into 10 consonant-family keys arranged in a compact grid. Each key produces five characters via five gestures: a center *tap* and four directional *flicks* (up, down, left, right). This design yields large touch targets, single-hand operability, and a one-to-one mapping between gesture and character—no multi-tap ambiguity.

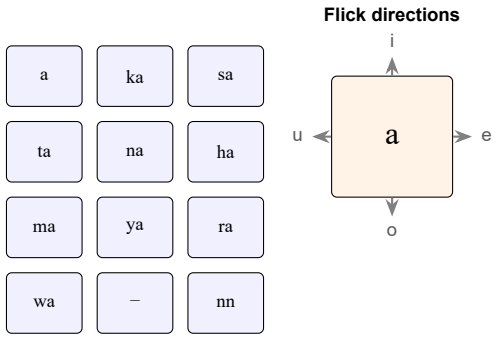

Figure 1: The Japanese 12-key flick keyboard groups 46 hiragana into consonant-family keys. Each key maps 5 characters to a tap and four flick directions (right callout). PadPim adapts this paradigm for Thai script.

Despite the success of flick input for Japanese, **to our knowledge no prior work has adapted this paradigm to Thai script**. Thai, with 69 characters, is a natural candidate: the character count is comparable to Japanese kana, and the characters can be partitioned into groups that fit a 12-key × 5-direction grid (60 slots on the main page).

In this paper we present **PadPim**[1], a data-driven 12-key flick keyboard for Thai. Our contributions are:

1. **Thai flick keyboard**: to our knowledge, the first application of the 12-key flick paradigm to Thai script, achieving 97.5% one-action character coverage.
2. **Data-driven layout pipeline**: we collect character frequencies from over one million lines of modern Thai conversation, derive a position-scoring model, and apply a greedy optimization algorithm to assign characters to key positions.
3. **Two layout variants**: *PadPim-Opti* maximizes typing efficiency via pure frequency optimization; *PadPim-Key* groups vowels onto dedicated keys for faster learning.
4. **Open-source implementation**: PadPim is released as an Android input-method editor, currently in public beta.

2. Background and Related Work

2.1 Thai Script and Existing Input Methods

Thai is an abugida written left-to-right without inter-word spaces [1]. Its 44 consonants represent initial and final consonant sounds; 21 vowel forms combine as prefixed, suffixed, superscript, or subscript glyphs around a base consonant; and

[1] From Thai แพด (*pad*, touchpad) + พิมพ์ (*pim*, to type).

4 tone marks (ไม้เอก ไม้โท ไม้ตรี ไม้จัตวา) are placed above the consonant to indicate lexical tone [1].

The **Kedmanee** layout, the *de facto* standard, was designed by Sawasdee Sukkasame in 1968 for the IBM Selectric typewriter. It distributes 69 characters across 47 physical keys with a two-layer shift mapping. While adequate for full-size keyboards, the layout was never optimized for frequency or ergonomics.

The **Pattachote** layout, proposed by Sarakit Pattachote in 1986, rearranges the character assignments based on frequency analysis and places high-frequency characters on the home row. Despite demonstrated efficiency gains, Pattachote never achieved mainstream adoption due to the installed base of Kedmanee.

On mobile, Thai users predominantly type on miniaturized Kedmanee QWERTY keyboards provided by Gboard, Samsung Keyboard, and SwiftKey. These layouts inherit all the desktop-era limitations—small keys, frequent shifting, two-handed design—compounded by the constraints of a 6-inch touchscreen.

2.2 Flick Input on 12-Key Keypads

The 12-key phone keypad has been used for text input since the advent of SMS. Early approaches used *multi-tap*—pressing a key multiple times to cycle through associated characters—and *T9* predictive disambiguation [7, 12]. Both suffer from ambiguity or multi-press overhead.

The **flick keyboard** eliminates these issues by mapping each character to a unique gesture: a center tap or a directional flick (up, down, left, right) on one of 10–12 keys. Introduced on the Apple iPhone in 2008 for Japanese *kana* input and popularized by Google Japanese Input, flick keyboards have become the dominant mobile input method in Japan.

Flick input offers several advantages over QWERTY on mobile: (1) *large touch targets*, since 12 keys occupy the same screen area as 30+ QWERTY keys; (2) *one-handed operability*, since the compact grid fits within the thumb's functional reach [9, 3]; (3) *unambiguous input*, since every character maps to exactly one gesture.

To our knowledge, the paradigm has not been applied to Thai.

2.3 Keyboard Layout Optimization

Layout optimization for touchscreen keyboards is well-studied. Fitts' law [6] predicts that movement time to a target depends on its distance and width; smaller, farther keys are slower to reach. Zhai et al. [15] optimized QWERTY-like layouts by minimizing Fitts'-law movement costs between character pairs. Bi et al. [4] extended this to multilingual keyboards. Oulasvirta et al. [11] used combinatorial optimization to design the KALQ split-thumb keyboard. Dunlop and Levine [5] applied multi-objective Pareto optimization balancing speed, familiarity, and error correction.

These prior works focus on QWERTY-like layouts with one character per key. In contrast, our work optimizes a *flick keyboard* where each key maps five characters via directional gestures, introducing a second dimension—flick-direction difficulty—into the scoring model.

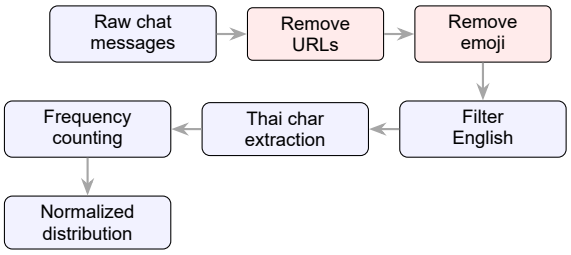

Figure 2: Data processing pipeline. Raw conversation text is filtered to remove non-Thai content, then Thai characters are extracted and counted to produce a normalized frequency distribution.

3. Data Collection and Character Frequency Analysis

3.1 Corpus Collection

To design a keyboard layout optimized for modern Thai communication, we collected character frequency data from real conversational text rather than relying solely on published corpora from formal sources.

We recruited **138 Thai university students** (age range 18–25) who voluntarily contributed anonymized character frequency counts extracted from their personal messaging conversations. Source platforms included LINE, Facebook Messenger, WhatsApp, Discord, and Twitter (now X) direct messages. The resulting corpus comprises over **1,000,000 lines** of Thai conversational text.

Ethics and privacy. To protect participant privacy, we employed a two-stage anonymization protocol: (1) participants ran a local character-counting script on their own devices, producing only aggregated per-character frequency counts; (2) only these anonymous numerical counts were transmitted to the researchers—no raw message text was ever collected or stored. Informed consent was obtained from all participants.

3.2 Processing Pipeline

The processing pipeline (Figure 2) operates as follows:

1. **URL and link removal**: Regular expressions strip HTTP(S) links and shortened URLs.
2. **Emoji and sticker removal**: Unicode emoji ranges (U+1F600–U+1F9FF, etc.) are filtered out.
3. **English and number filtering**: Non-Thai script (Latin, digits, CJK) is removed based on Unicode block detection (Thai block: U+0E00–U+0E7F).
4. **Thai character extraction**: Remaining text is segmented into individual Thai characters, distinguishing base consonants, vowel forms, tone marks, and diacritics [14].
5. **Frequency counting**: Per-character absolute counts are computed across all participants, then normalized to percentages.

A combination of automated scripts and manual verification was used to ensure data quality, particularly for edge cases involving mixed-script messages and informal transliterations.

3.3 Frequency Results

Table 1 presents the 30 most frequent Thai characters in our corpus. The most frequent character is า (*sara aa*) at 7.12%, followed by ร (*ro rua*) at 6.49% and น (*no nu*) at 6.08%.

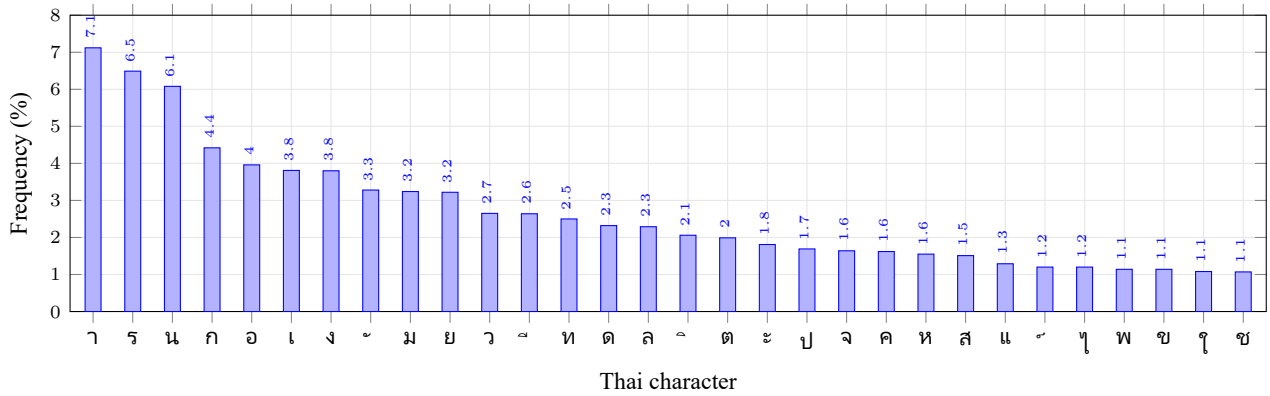

Figure 3: Character frequency distribution for the top 30 Thai characters in our conversational corpus. The distribution exhibits a Zipf-like curve: the top 10 characters account for 45.4% of all Thai character usage, while the top 30 cover 82.8%.

Validation against published data. We compared our conversational frequency distribution against the formal-text frequencies published by Aroonmanakun [2], who analyzed a corpus of approximately 354 million Thai characters from mixed published sources. The Spearman rank-order correlation between the two distributions is $\rho = 0.97$ ($p < 0.001$), with the top 10 characters appearing in nearly identical order. This strong concordance suggests that Thai character frequencies are relatively stable across registers (formal vs. informal) and time periods (2006 vs. 2024), validating the use of modern conversational data for layout design.

The frequency distribution exhibits a Zipf-like shape (Figure 3): the top 10 characters account for 45.4% of all usage, and the top 30 cover 82.8%. This heavy concentration at the top of the distribution is precisely what makes a 12-key flick layout viable—the majority of typing involves a small subset of characters that can be placed on the most accessible key positions.

4. Layout Design Methodology

Our layout design proceeds in four stages: (1) define the grid structure, (2) score each position by ergonomic accessibility, (3) assign characters to positions via a greedy algorithm, and (4) handle special cases (tone marks, rare characters).

4.1 Grid Structure

PadPim arranges **12 character keys** in a 3×4 grid (3 columns, 4 rows), flanked by utility keys on each side (Figure 4). The left column holds input-mode controls (microphone, cursor-left, language switch, shift/emoji); the right column holds action keys (backspace, cursor-right, space, enter). This yields a full 5×4 grid on the keyboard surface, with the central 3×4 block dedicated to character input.

Each character key supports **5 input gestures**: a center *tap* and four directional *flicks* (up, down, left, right). The bottom row (row 3) omits the down-flick because keys at the screen edge have no room for downward gesture travel, yielding:

$$N_{\text{slots}} = 3 \times 3 \times 5 + 3 \times 1 \times 4 = 45 + 12 = 57$$

Thus, 57 characters can be placed on the main page without any shift layer.

4.2 Position Scoring Model

We assign each of the 57 character positions a **combined score** that reflects how easy it is for the user's thumb to reach. The score has two components: a *grid score* and a *flick multiplier*.

Grid score. Based on ergonomic thumb-reach studies [3, 9] and our own week-long empirical testing with a prototype, we assigned grid scores to each key position (Figure 4). The center key of the middle row (row 1, col 2) receives the highest score of 100—it lies at the natural thumb resting position. Adjacent mid-row keys score 80; top-row and lower-row keys score 40–60; the bottom row scores 20.

Flick multiplier. Not all flick directions are equally easy. Through empirical testing with a prototype over one week and informed by Japanese flick-keyboard ergonomics research, we determined the following multipliers:

- **Tap** (center): 1.0 — the default, easiest action
- **Up**: 0.8 — natural upward thumb extension
- **Left / Right**: 0.6 — lateral thumb movement
- **Down**: 0.4 — least natural; opposes gravity for the thumb

Combined score. For a key at grid position (r, c) with flick direction d:

$$S(r, c, d) = G(r, c) \times M(d) \tag{1}$$

where $G(r, c)$ is the grid score and $M(d)$ is the flick multiplier. For example, a tap on the center key scores $100 \times 1.0 = 100$, while a left-flick on a bottom-row key scores $20 \times 0.6 = 12$.

4.3 Greedy Assignment Algorithm

Given the 57 scored positions and 57 characters sorted by frequency, we apply a **greedy assignment** (Algorithm 1): the highest-frequency character is assigned to the highest-scoring position, the second-highest to the second-highest position, and so on.

This greedy approach is provably optimal under the assumption that each character is typed independently (i.e., no bigram transition costs). By the *rearrangement inequality* [8], pairing the two sequences in matching sorted order maximizes $\sum_i f_i \cdot s_i$, the frequency-weighted total score. Concretely: if $f_1 \geq f_2 \geq \cdots$ and $s_1 \geq s_2 \geq \cdots$, then for any permutation σ, $\sum_i f_i s_i \geq \sum_i f_i s_{\sigma(i)}$, so the identity

Table 1: Top 30 Thai characters by frequency in our corpus of 1M+ lines of conversational text from 138 participants.

Rank	Char	Name	Type	Freq %
1	า	Sara aa	Vowel	7.12
2	ร	Ro rua	Cons.	6.49
3	น	No nu	Cons.	6.08
4	ก	Ko kai	Cons.	4.42
5	อ	O ang	Cons.	3.96
6	เ	Sara e	Vowel	3.81
7	ง	Ngo ngu	Cons.	3.80
8	ั	Sara an	Vowel	3.28
9	ม	Mo ma	Cons.	3.24
10	ย	Yo yak	Cons.	3.22
11	ว	Wo waen	Cons.	2.65
12	ี	Sara ii	Vowel	2.64
13	ท	Tho thahan	Cons.	2.50
14	ด	Do chada	Cons.	2.32
15	ล	Lo ling	Cons.	2.29
16	ิ	Sara i	Vowel	2.06
17	ต	To tao	Cons.	1.99
18	ะ	Sara a	Vowel	1.81
19	ป	Po pla	Cons.	1.69
20	จ	Jo jan	Cons.	1.64
21	ค	Kho khwai	Cons.	1.62
22	ห	Ho hip	Cons.	1.55
23	ส	So suea	Cons.	1.51
24	แ	Sara ae	Vowel	1.29
25	์	Thanthakhat	Diacr.	1.20
26	ไ	Sara ai mai	Vowel	1.20
27	พ	Pho phan	Cons.	1.14
28	ข	Kho khai	Cons.	1.14
29	ใ	Sara ai muan	Vowel	1.08
30	ช	Cho chang	Cons.	1.07

permutation—the greedy zip—is optimal under the independence assumption.

4.4 Special Handling

Tone marks on the space key. The three most common tone marks—◌่ (*mai ek*, 2.50%), ◌้ (*mai tho*, 1.80%), and ◌๊ (*mai tri*, 0.15%)—are placed on the *space key* as left, up, and right flicks, respectively. This design is motivated by the observation that tone marks almost always follow a consonant; after typing the preceding consonant with the left hand area, the thumb naturally returns toward the space bar region. Placing tones on the space key frees three valuable main-grid positions for consonants and vowels.

Shift page for rare characters. The 9 rarest Thai characters—ฆ (0.03%), ◌ฺ (0.02%), ◌ฺ (0.01%), ฌ (0.01%), ฟ (0.01%), ฮ (<0.01%), ซ, ฑ, ฎ (≈0%)—are relegated to a shift layer accessible via a dedicated shift key. These characters collectively account for only ∼0.1% of Thai text, meaning the shift key is almost never needed in practice.

Coverage summary. These three character sets are mutually exclusive—no character appears on more than one layer:

- Main page: 57 characters (∼93% of Thai character occurrences)
- Space-key tones: 3 tone marks (∼4.5%)
- Shift page: 9 rare characters (∼0.1%)

Together, the main page and space-key tones account for approximately 97.5% of Thai character occurrences. All 69 Thai characters are accessible across the three layers.

Figure 4: Position scoring heatmap for the 5 × 4 keyboard grid. The central 3×4 block contains character keys scored by ergonomic thumb reach: 100 (center, optimal), 80 (mid-row), 60/40 (top/lower rows), 20 (bottom row). Utility keys (gray) flank each side.

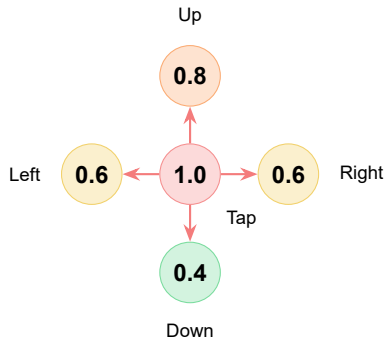

Figure 5: Flick direction multipliers. The tap (center) is easiest (1.0); upward flick (0.8) is natural for the thumb; lateral flicks (0.6) require more effort; downward flick (0.4) is hardest.

5. Layout Variants

We present two layout variants that trade off between raw typing efficiency and learnability.

5.1 PadPim-Opti: Pure Frequency Optimization

PadPim-Opti is the direct output of the greedy algorithm: every character is placed strictly according to its frequency-to-score rank. Figure 6 shows the complete layout.

The highest-frequency character, า (7.12%), occupies the center tap of the highest-scoring key (row 1, col 2, score 100). The second-highest, ร (6.49%), takes the up-flick of the same key (score 80). Characters are distributed across all 12 keys purely by the greedy assignment.

Advantage: PadPim-Opti achieves the theoretically maximum frequency-weighted position score—it is the fastest layout for experienced users.

Disadvantage: vowels and consonants are intermixed across keys with no categorical grouping, making the layout harder to learn initially.

5.2 PadPim-Key: Vowel-Grouped Variant

PadPim-Key addresses learnability by consolidating all vowels onto **two dedicated keys** (สระ ๑ and สระ ๒, meaning "Vowel 1" and "Vowel 2"), located at the center of the grid. The remaining 10 keys hold consonants, assigned by greedy optimization within the consonant subset.

Algorithm 1: Greedy Character Assignment

Input: Characters $C = \{c_1, \ldots, c_{57}\}$ with frequencies f_i
;
Positions $P = \{p_1, \ldots, p_{57}\}$ with scores s_j
Output: Assignment $A : C \to P$

```
Sort C by descending frequency: f_1 ≥ f_2 ≥ ··· ;
Sort P by descending score: s_1 ≥ s_2 ≥ ··· ;
for i = 1 to 57 do
    A(c_i) ← p_i;
return A;
```

Row 0 (low priority)		
ข ธ ะ ผ ื	ล ห ม ส ่	ใ ั ป ี ฟ
Row 1 (highest priority)		
อ ว น ี ไ	ร ง า ้ ต	เ ท ก ด พ
Row 2 (low priority)		
ช ำ จ ณ ญ	ิ แ ย ์ บ	็ ศ ค ถ ภ
Row 3 (super-low, no down-flick)		
ๆ ฯ โ ฎ	ฉ ฏ ซ ฐ	ฒ ฤ ษ ฑ

Figure 6: PadPim-Opti layout. Each cell shows: *top*=up-flick, *left* **center** *right*=left-flick / tap / right-flick, *bottom*=down-flick. Bold center character is the tap (highest priority per key). Characters are assigned purely by frequency-score matching.

- **Vowel Key 1** (row 1, col 2): า,ั, เ,ี, ะ—the five most frequent vowels.
- **Vowel Key 2** (row 2, col 2): ิ,ุ, แ,ู, ำ—the next five vowels.

Advantage: Users learn that "vowels are always on these two pink keys," dramatically reducing visual search time for beginners.

Disadvantage: Grouping constrains the optimizer. Some vowels are forced onto lower-scoring positions than they would occupy in the unconstrained (Opti) layout.

5.3 Variant Comparison

Table 2 compares the two variants on frequency-weighted metrics. We compute the *weighted average score* as $\bar{S} = \sum_i f_i \cdot S_i$, where f_i is the frequency and S_i the position score of character i.

PadPim-Opti achieves a 6.3% higher weighted average score than PadPim-Key. However, PadPim-Key's categorical vowel grouping substantially reduces the cognitive search space: instead of scanning all 12 keys for a vowel, users need only check two fixed locations. We hypothesize that PadPim-Key will exhibit a faster learning curve, while PadPim-Opti will yield higher peak performance for experienced users.

6. Theoretical Evaluation

In the absence of a formal user study—planned as future work (§8)—we evaluate PadPim using theoretical efficiency metrics and compare against two QWERTY baselines.

Row 0 (low priority)		
่ ื ล บ ี	ด ค ง ห ช	พ ธ ต ผ ฟ
Row 1 (highest priority)		
ก ม ร ย ส	เ ั า ี V1 ะ	อ ว น ท ไ
Row 2 (low priority)		
ข ื ป ณ ญ	แ ิ V2 ำ	ใ ศ จ ถ ภ
Row 3 (super-low, no down-flick)		
ๆ ฯ โ ฎ	ฉ ฏ ซ ฐ	ฒ ฤ ษ ฑ

Figure 7: PadPim-Key layout. The two dedicated vowel keys are marked ^{V_1} and ^{V_2} (shaded in color prints). All vowels are concentrated on these keys for faster learning; consonants fill the remaining 10 keys by frequency optimization.

Table 2: Comparison of PadPim-Opti and PadPim-Key layout variants. Higher weighted score indicates better average ergonomic placement.

Metric	PadPim-Opti	PadPim-Key
One-action coverage	97.5%	97.5%
Characters on main page	57	57
Weighted avg. score	62.4	58.7
Top-10 char avg. score	76.8	73.2
Vowel avg. score	55.1	50.8
Consonant avg. score	65.3	62.9
Vowel keys	Distributed	2 dedicated
Learning model	Memorize all	Learn 2 vowel keys

6.1 Baselines

- **Kedmanee**: Standard Thai QWERTY layout. 47 keys, 2 shift layers. Used by the vast majority of Thai typists.
- **Pattachote**: Frequency-optimized Thai QWERTY layout. 47 keys, 2 shift layers. Positions high-frequency characters on the home row.

Both baselines are designed for full-size keyboards and assume two-handed typing. On mobile, they are rendered as miniaturized touchscreen QWERTY layouts.

6.2 Comparison Metrics

Table 3 presents the comparison.

Keys per character (KPC). KPC measures the average number of key actions required to produce one Thai character [13, 10]. We restrict the calculation to the 69 Thai characters (consonants, vowels, tone marks, and diacritics), excluding Arabic numerals and punctuation, which are handled by separate keyboard modes in all layouts compared. For PadPim, the 60 one-action characters (57 main-page + 3 space-key tones) account for a fraction $w_1 = 0.975/0.976 \approx 0.999$ of Thai character occurrences, and the 9 shift-page characters account for $w_2 \approx 0.001$:

$$\text{KPC}_{\text{PadPim}} = 0.999 \times 1.0 + 0.001 \times 2.0 \approx 1.001$$

For Kedmanee, we mapped our corpus onto the standard

Table 3: Theoretical comparison of PadPim against Kedmanee and Pattachote baselines on a 6-inch (68 mm wide) mobile display. Baseline KPC and shift-frequency values were derived by mapping our corpus onto each layout's key-to-character assignment and counting one-action vs. shifted character occurrences.

	PadPim-O	PadPim-K	Kedmanee	Pattachote
Total keys	12	12	47	47
Shift layers	1	1	2	2
Chars/key (max)	5	5	1	1
One-hand capable	Yes	Yes	No	No
Coverage & efficiency				
One-action coverage	97.5%	97.5%	∼55%	∼55%
Avg. KPC	1.001	1.001	1.29	1.25
Shift freq. /100 chars	0.1	0.1	∼29	∼25
Touch target (6-inch screen)				
Key width (mm)	13.6	13.6	5.8	5.8
Key area (mm^2)	184	184	44	44
Area ratio	4.2×	4.2×	1×	1×

two-layer layout and counted the fraction of Thai character occurrences that fall on the shifted layer. Approximately 29% require the shift key (including common tone marks and several vowels), yielding $\text{KPC}_{\text{Kedmanee}} \approx 1.29$. The same procedure on the Pattachote layout gives $\text{KPC}_{\text{Pattachote}} \approx 1.25$.

Touch target area. On a typical 6-inch smartphone (68 mm usable keyboard width), PadPim's 12 character keys in 3 columns each span approximately 13.6 mm—more than twice the 9.2 mm minimum recommended by Bergström-Lehtovirta and Oulasvirta [3]. In contrast, Kedmanee's 10-column QWERTY layout yields keys only ∼5.8 mm wide, well below the ergonomic threshold. PadPim's keys are **4.2× larger** in area.

One-handed operation. PadPim's 3×4 grid is specifically designed for single-thumb operation: all keys fall within the thumb's functional arc [9]. QWERTY layouts require either two-thumb typing or slow single-finger hunt-and-peck across a wide keyboard.

6.3 Fitts' Law Analysis

Fitts' law [6] models movement time as:

$$\text{MT} = a + b \cdot \log_2\left(\frac{D}{W} + 1\right)$$

where D is the distance to the target and W is the target width. For PadPim, the effective D is small (keys are adjacent, maximum thumb travel ∼30 mm) and W is large (13.6 mm), yielding a low index of difficulty. For QWERTY, D can exceed 50 mm (opposite corners) while W is only 5.8 mm. We estimate that PadPim's average Fitts' index of difficulty is approximately 40–60% lower than QWERTY, suggesting substantially faster inter-key movement—though actual typing speed also depends on visual search time and motor learning, which require empirical measurement.

6.4 Limitations

Our theoretical evaluation has several limitations:

1. **No empirical typing data**: We have not yet conducted a formal user study measuring typing speed (WPM), error rate, or learning curve.
2. **Flick multipliers**: The direction multipliers (1.0, 0.8, 0.6, 0.4) were determined from one week of prototype testing by the first author, supplemented by Japanese ergonomics literature. A larger empirical study would strengthen these values.
3. **No bigram optimization**: The greedy algorithm treats characters independently. It does not penalize placing frequently co-occurring characters (e.g., กา, รา) on the same key, where consecutive same-key flicks may be slower.
4. **Corpus bias**: Our corpus consists of university students aged 18–25 communicating via messaging apps. Professional, literary, or formal Thai text may exhibit different frequency distributions, though our validation against Aroonmanakun's formal corpus [2] suggests the differences are small.

7. Implementation

PadPim is implemented as an Android Input Method Editor (IME) in Kotlin, targeting Android 7.0+ (API 24). The application is open-source and currently in public beta testing.

Architecture. The core service, `ThaiFlickIMEService`, extends Android's `InputMethodService` and manages keyboard mode switching (Thai, English, Emoji, Numbers, Shift), text suggestions, and speech recognition. The keyboard UI is rendered by `FlickKeyboardView`, a custom `View` that draws the 5×4 key grid on a Canvas and handles touch input directly.

Gesture detection. The `FlickGestureDetector` class implements flick recognition using a dead-zone model: (1) on touch-down, the origin point is recorded; (2) on touch-move, the Euclidean distance from the origin is computed; (3) if the distance exceeds a configurable dead-zone radius (default: 20 dp), the flick direction is determined from the angle $\theta = \text{atan2}(-\Delta y, \Delta x)$, partitioned into four 90 quadrants; (4) if the distance remains within the dead zone, the gesture is classified as a tap. The dead-zone radius is user-configurable via a sensitivity slider in the settings.

Features. Beyond the core flick input, PadPim includes:

- *Word suggestions*: prefix-matching against a dictionary of the top 500 Thai words, displayed in a suggestion bar above the keyboard.
- *Theme system*: light and dark modes with Material Design colors.
- *Haptic feedback*: configurable vibration on key press.
- *Clipboard history*: stores up to 20 recent clipboard entries.
- *Speech-to-text*: integrated Android `SpeechRecognizer` for Thai and English voice input.
- *Interactive tutorial*: a 7-step onboarding tutorial with animated finger demonstrations, a position-scoring heatmap, and a typing practice exercise.
- *Layout customization*: advanced users can remap individual keys via a graphical editor.

Privacy. PadPim requests only `VIBRATE` and `RECORD_AUDIO` permissions. It does *not* request the

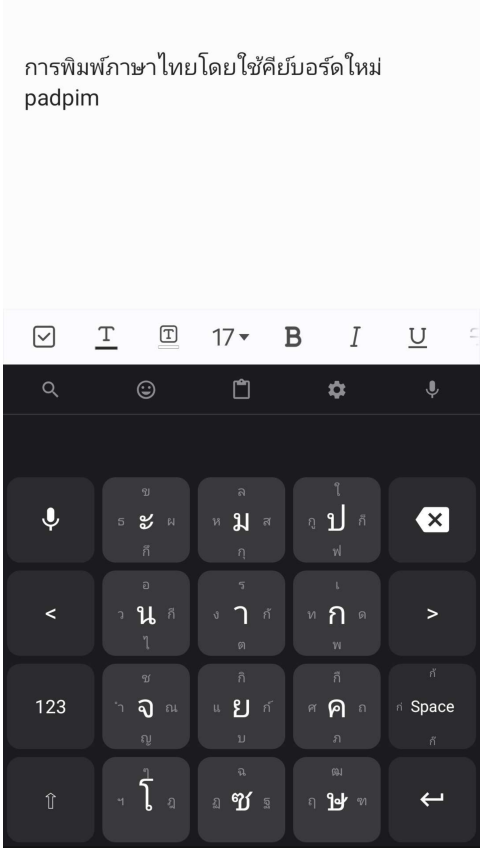

Figure 8: PadPim Android application showing the PadPim-Opti layout in dark theme. The 3×4 character grid occupies the center, flanked by utility keys. Each key displays the tap character prominently with flick-direction hints in smaller text at the edges.

`INTERNET` permission: no telemetry is collected and no analytics are included. All keyboard and preference data remains on-device. The optional speech-to-text feature delegates to the Android system `SpeechRecognizer`, whose network behavior is governed by the device manufacturer's implementation (e.g., Google's cloud speech service); PadPim itself neither initiates nor controls any network traffic.

8. Discussion and Future Work

User study. The most critical next step is a formal within-subjects user study comparing PadPim-Opti, PadPim-Key, and Kedmanee QWERTY on: (1) typing speed (WPM), (2) character error rate (CER), (3) subjective preference (NASA-TLX workload assessment), and (4) learning curve over a 2–4 week longitudinal period. We plan to recruit 30+ participants with varying Thai keyboard experience to measure both novice onboarding and expert performance.

Bigram-aware optimization. The current greedy algorithm optimizes for single-character frequency. A natural extension is to incorporate bigram transition costs—penalizing the placement of frequently co-occurring characters on the same key or on distant keys. Metaheuristic approaches (simulated annealing, genetic algorithms) could search the larger combinatorial space that arises when bigram effects are considered [15, 5].

Diagonal flick directions. The current design uses 4 flick directions (up, down, left, right). Adding diagonal directions (up-left, up-right, down-left, down-right) would expand each key to 9 input slots, potentially accommodating more characters on the main page. However, diagonal flicks may reduce accuracy; this trade-off merits empirical investigation.

Personalized layouts. Individual users may exhibit different character frequency distributions depending on their communication style, profession, or dialect. PadPim already supports a custom layout editor; a future version could automatically generate a personalized optimal layout from a user's own typing history.

Cross-platform deployment. PadPim is currently Android-only. Porting to iOS (via a custom keyboard extension) and web-based virtual keyboards would broaden the potential user base.

Accessibility. The large touch targets of a 12-key layout may benefit users with motor impairments. We plan to investigate PadPim's usability for accessibility use cases in collaboration with occupational therapists.

9. Conclusion

We have presented PadPim, to our knowledge the first data-driven 12-key flick keyboard designed for Thai script. By collecting character frequencies from over one million lines of modern Thai conversation, defining a position-scoring model that accounts for both grid ergonomics and flick-direction difficulty, and applying a greedy assignment algorithm, PadPim maps 57 Thai characters onto 12 keys. Together with three tone marks on the space key, approximately 97.5% of Thai character occurrences are reachable without a shift layer.

Our results suggest that the Japanese flick-input paradigm transfers effectively to Thai despite the script's 69-character inventory. The two layout variants—PadPim-Opti for maximum speed and PadPim-Key for easier learning—offer users a meaningful choice. Compared to the standard Kedmanee QWERTY layout on mobile, PadPim offers $4.2\times$ larger touch targets, near-unity keys-per-character, and true one-handed operability.

The primary limitation of this work is the absence of a formal user study; the evaluation relies on theoretical metrics. The open-source Android implementation is available for public beta testing, and we plan a within-subjects user study to empirically validate the efficiency gains reported here.

Acknowledgments

We thank the 138 university student volunteers who contributed aggregated character-frequency counts for this research.

Data and Code Availability

The PadPim Android application source code is publicly available at `https://github.com/Jnx03/ThaiFlickKeyboard`. The character frequency data used for layout optimization and the layout specifications are documented in the repository under `docs/layout.md`. Raw conversation text was not retained; only anonymized, aggregated character counts were collected and used.

References

[1] Wirote Aroonmanakun. Collocation and Thai word segmentation. In *Proceedings of the SNLP-Oriental COCOSDA Workshop*, pages 68–75, 2002.

[2] Wirote Aroonmanakun. List of Thai characters sorted by frequency. Department of Linguistics, Faculty of Arts, Chulalongkorn University, 2006.

[3] Jennifer R. Bergstrom-Lehtovirta and Antti Oulasvirta. Modeling the functional area of the thumb on mobile touchscreen surfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 1991–2000. ACM, 2014.

[4] Xiaojun Bi, Barton A. Smith, and Shumin Zhai. Multilingual touchscreen keyboard design and optimization. *Human–Computer Interaction*, 27(4):352–382, 2012.

[5] Mark D. Dunlop and John Levine. Multidimensional Pareto optimization of touchscreen keyboards for speed, familiarity and improved spell checking. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 2669–2678. ACM, 2012.

[6] Paul M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47:381–391, 1954.

[7] Dale L. Grover, Martin T. King, and Clifford A. Kushler. A reduced keyboard disambiguating computer. U.S. Patent No. 5,818,437, 1998.

[8] Godfrey H. Hardy, John E. Littlewood, and George Pólya. *Inequalities*. Cambridge University Press, 2nd edition, 1952.

[9] Amy K. Karlson, Benjamin B. Bederson, and Jose L. Contreras-Vidal. Understanding single-handed mobile device interaction. Technical Report HCIL-2006-02, Human-Computer Interaction Lab, University of Maryland, College Park, 2006.

[10] I. Scott MacKenzie and R. William Soukoreff. Text entry for mobile computing: Models and methods, theory and practice. *Human–Computer Interaction*, 17(2–3):147–198, 2002.

[11] Antti Oulasvirta, Anna Reichel, Wenbin Li, Yan Zhang, Myroslav Bachynskyi, Keith Vertanen, and Per Ola Kristensson. Improving two-thumb text entry on touchscreen devices. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 2765–2774. ACM, 2013.

[12] Miika Silfverberg, I. Scott MacKenzie, and Panu Korhonen. Predicting text entry speed on mobile phones. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 9–16. ACM, 2000.

[13] R. William Soukoreff and I. Scott MacKenzie. Metrics for text entry research: An evaluation of MSD and KSPC, and a new unified error metric. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 113–120. ACM, 2003.

[14] Thanaruk Theeramunkong and Sasiporn Usanavasin. Non-dictionary-based Thai word segmentation using decision trees. In *Proceedings of the First International Conference on Human Language Technology Research*. ACL, 2001.

[15] Shumin Zhai, Michael Hunter, and Barton A. Smith. Performance optimization of virtual keyboards. *Human–Computer Interaction*, 17(2–3):229–269, 2002.